

Beginner's Guide: The Dynamic "NoSQL" Database in C

1. The Core Concept: "Forms" vs. "Blank Paper"

To understand this code, imagine two ways of filing information.

The Old Way (Structured/SQL)

Imagine a pre-printed government form. It has specific boxes: [Name], [Age], [ID].

- **Pro:** It's organized.
- **Con:** If you want to write down your "Favorite Color," there is no box for it. You are stuck with the boxes provided.
- *In C terms:* This is a standard struct with fixed arrays (char name[50]).

The New Way (Unstructured/NoSQL - This Code)

Imagine a **blank sheet of paper**.

- You can write "Name: John".
- On the next line, you can write "Hobby: Fishing".
- On the next sheet, for a different person, you might write "Car Model: Toyota" and "Year: 2020".
- **Pro:** You can store *anything*.
- **Con:** You need to manage the layout yourself.
- *In C terms:* This is what our code does using **Dynamic Memory**.

2. The "Building Blocks" (Structs)

The code uses two main structures. Think of them like this:

A. The "Sticky Note" (Field)

In our code, a single piece of data (like Name = John) is called a **Field**.

```
typedef struct {  
    char *key; // The label (e.g., "Name")  
    char *value; // The content (e.g., "John")  
} Field;
```

- **Real Life Analogy:** A single sticky note where you write a label and a value.
- **Why char *?** We use pointers (asterisks) because we don't know how long the text will be. It could be "Id=1" (short) or "Description=A very long string..." (long). Pointers allow

the text to be any size.

B. The "Page" (Document)

A document is a collection of sticky notes.

```
typedef struct {  
    Field *fields;    // A list of Sticky Notes  
    int field_count;  // How many notes are on this page?  
} Document;
```

- **Real Life Analogy:** A single sheet of paper. You can stick as many "Sticky Notes" (Fields) on it as you want.
- **Field *fields:** This is a pointer to the *list* of notes. Initially, the page is blank (NULL). As you add notes, we grow this list.

3. The "Magic Backpack" (Dynamic Memory)

In C, normally you have to decide storage size upfront (e.g., "I need space for 100 numbers"). But what if you don't know how much data the user has?

We use three special tools:

1. **malloc (Memory Allocation):** Asking the computer, "Give me a brand new empty box."
2. **realloc (Re-Allocation):** Asking the computer, "This box is full. Can you move my stuff to a bigger box?"
3. **free:** Telling the computer, "I'm done with this box, you can throw it away."

How create_document() works (Step-by-Step)

1. The User Starts:
The user says "I want to add a record."
2. **Expanding the Folder:**
`Document *temp = (Document *)realloc(db, (doc_count + 1) * sizeof(Document));`
 - *Analogy:* You have a folder of pages. It's full. You buy a slightly thicker folder that fits exactly **one more page** than before.
3. Adding Sticky Notes (The Loop):
The code enters a while(1) loop. It asks the user for a Key and Value.
`Field *f_temp = (Field *)realloc(db[doc_count].fields, (count + 1) * sizeof(Field));`
 - *Analogy:* You are holding the page. You write a new sticky note. You realize the page is too small. You magically stretch the page to fit exactly **one more note**.
4. **Saving the Text:**
`db[doc_count].fields[count].key = key;`

- The key variable is a pointer to the text the user typed. We glue that text onto our "expanded page."

4. Real Life Scenario: The "General Store"

Imagine you run a store that sells **everything**. You need a database.

Customer 1 buys an Apple:

- You create a Document.
- Field 1: Type = Fruit
- Field 2: Color = Red
- Field 3: Price = \$1.00

Customer 2 buys a Laptop:

- You create a Document.
- Field 1: Type = Electronics
- Field 2: Brand = Dell
- Field 3: CPU = i7
- *Note:* This document has totally different fields than the Apple!

The Code in Action:

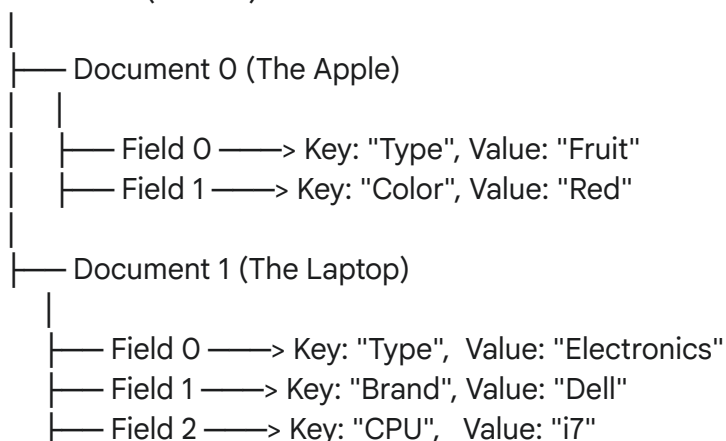
If this was a standard C array (struct { char color[20]; int cpu; }), you would have wasted space. The Apple doesn't have a CPU, and the Laptop doesn't have a flavor.

Our Dynamic/NoSQL approach handles this perfectly because every document carries only the specific fields it needs.

5. Visualizing the Memory Structure

When the program runs with the example above, memory looks like this:

DATABASE (Pointer)



6. Summary of Key C Functions Used

Function	Real Life Translation
malloc	"Give me a notepad."
realloc	"This notepad is full, give me a bigger one and copy my notes over."
free	"Burn this notepad, I don't need it anymore."
strdup	"Make a photocopy of this text."
struct	"A template for organizing information."